

One Size Fit All(pg_rust)

场景

优先级--高：

- 多模态数据模型：关系 + 向量 + 文档 + 图 + 时序 + KV；
- 混合检索：向量相似度 + 全文 + 结构化过滤 + 图遍历，单查询完成；
- ACID + 长事务 + Checkpoint；
- 事件溯源：仅追加日志 + 物化视图；
- HTAP：行存 OLTP + 列存 OLAP，自由路由；
- 自动向量化：INSERT 即 embedding，零 ETL；
- 向量压缩：SQ/PQ，自动策略选择；
- 审计血缘：行级日志 + 数据血缘 + 可解释性检索；
- MCP / LangGraph / OpenAPI 原生协议支持。

优先级--低：

- 细粒度安全：RLS + 动态脱敏 + 多租户配额；
- 分布式事务；
- 分层存储：NVMe -> S3 -> Glacier，查询透明；
- Serverless + 计算存储分离 + 秒级分支（COW）；
- 原生 LLM 推理：数据库内调用模型，批量执行。

解决的核心问题

现状：Agent 使用数据库的场景和方式是丰富且多样的，当前的解决方案是多个引擎拼在一起（PG + pgvector + ES + Neo4j + TimescaleDB）破坏了事务的原子性和崩溃一致性。

需要一种新的架构设计：对内核进行抽象，能让这些不同的需求共存于一个引擎内且不退化
成“在一个进程里跑 6 个数据库”，且这套架构是可扩展可持续演进的。

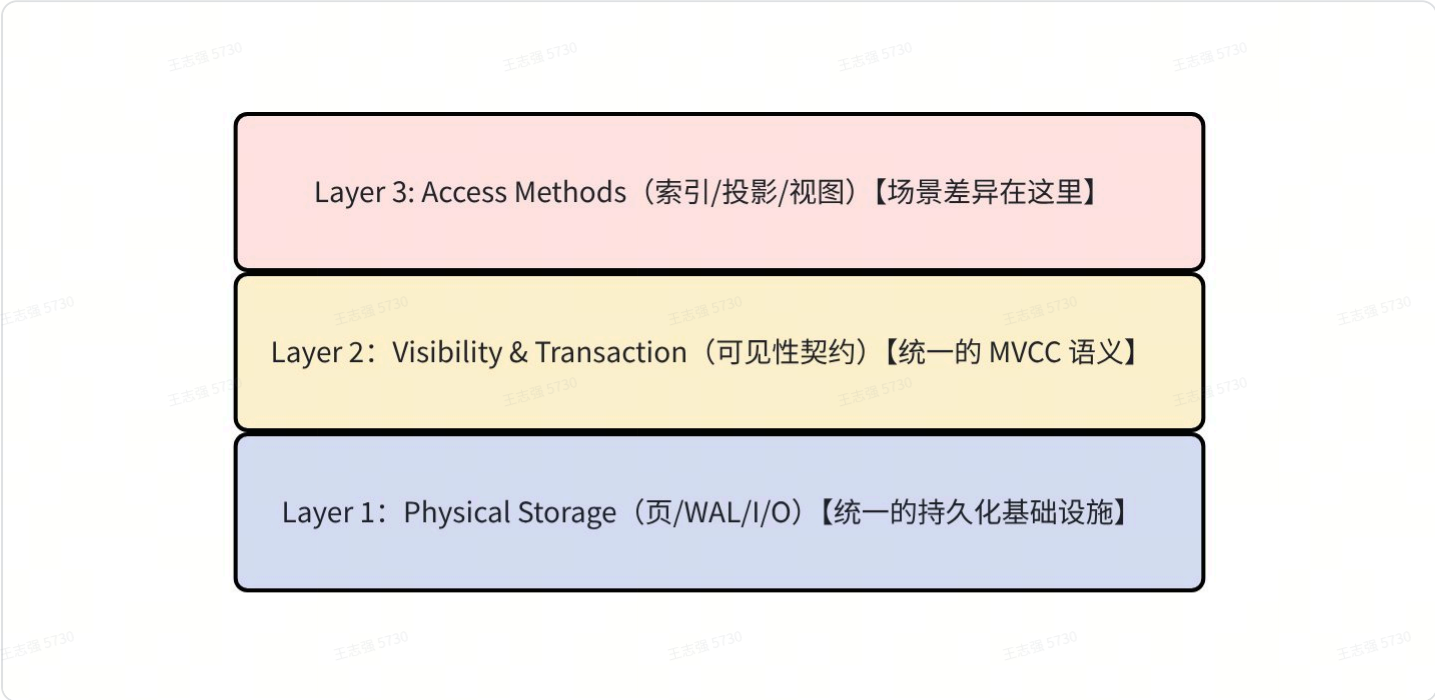
不同场景的不同特点：

数据形态	最优存储结构	典型引擎
------	--------	------

TP（OLTP 短事务）	行存 + B+Tree + 低延迟	Postgres, MySQL
AP（OLAP 分析）	列存 + 投影 + 向量化	DuckDB, ClickHouse
向量（语义检索）	图结构（HNSW）+ 大块连续内存	Milvus, Qdrant
全文（关键词搜索）	倒排表 + posting list	Elasticsearch
图（关系推理）	邻接表 + 属性存储	Neo4j, Memgraph
时序（行为轨迹）	时间分区 + 降采样 + TTL	TimescaleDB, InfluxDB

架构设计

从第一性原理出发，把存储内核分成三个正交层，每层独立演进：



Layer 1：物理存储层

所有场景共享。

该层提供场景无关的原语：

- Page Allocator：分配固定大小的页（8KB / 16 KB / 64 KB 可配置），不关心页里装什么；
- WAL Writer：append-only 日志，接受任意 `(record_type, payload)`，保证 fsync 语义；
- Buffer Pool：`page_id -> in-memory frame` 的映射 + eviction；
- LSN Clock：全局单调递增，所有组件共享；
- Checkpoint Coordinator：定期把 dirty page 持久化，截断 WAL。

关键设计：Page Allocator 不假设“页里面是行”。一个 page 可以是：

- 行存 slotted page (TP) ;
- 列存 compressed block (AP) ;
- HNSW 邻居列表 page;
- 倒排 posting list page;
- 图邻接表 page;
- 时序 chunk page。

它们在物理层面都是 “带 `page_lsn` 的 8 KB 块” ， Buffer Pool 统一管理，WAL 统一记录。

Layer 2：可见性与事务层

所有场景共享。

核心洞察：所有数据形态的 “源事实” 都是 Tuple（行），所有索引 / 投影都是 Tuple 的派生视图。
一个 Tuple 由 `TID (page_id, slot_id)` 唯一标识，由 `(xmin, xmax, snapshot)` 决定可见性。

Visibility Oracle——一个共享服务,任何访问方法都可以查询:

代码块

```
1 trait VisibilityOracle: Send + Sync {
2     fn is_visible(&self, xmin: TxId, xmax: TxId, snapshot: &Snapshot) -> bool;
3     fn current_snapshot(&self) -> Snapshot;
4     fn register_index(&self, index_id: IndexId, watermark_lsn: Lsn);
5     fn advance_watermark(&self, index_id: IndexId, applied_lsn: Lsn);
6 }
```

关键设计决策：索引条目不自己判断可见性，而是存 TID，回表后由 Visibility Oracle 判断。

索引	叶子条目
B+Tree	(key, TID)
HNSW	(vector, TID)
倒排	(term, posting: TID list + tf + position)
图邻接	(edge_id/TID, target_node_TID)
时序	(timestamp_bucket, TID list)

所有索引回表后，统一执行 `visibility_oracle.is_visible(tuple.xmin, tuple.xmax, snapshot)`

，不可见则跳过。

好处：

- 六种索引不需要各自实现 MVCC 逻辑；
- GC 统一：行版本被 vacuum 时，通知所有引用该 TID 的索引删除条目；
- 新增一种访问方法时，不需要重新实现事务语义。

代价与缓解：

- 回表开销（从索引拿到 TID 后要读行确认可见性）；
- 对 HNSW 等“近似”结构，会出现“索引返回 K 个候选，可见性过滤不足 K 个”的问题；
- 缓解：对 HNSW 多取 2x 候选，可见性过滤后 top-K，后续可升级为 TID + XID 索引条目，避免回表；
- 对 AP 列存投影：投影本身是最新快照的物化视图，不存旧版本，读取时不需要回表。

Layer 3: Access Methods

场景差异在这里。

这一层是可插拔的。每种方法都实现统一的 trait：

代码块

```
1 trait AccessMethod: Send + Sync {
2     type ScanState;
3     type Key;
4
5     fn am_type(&self) -> AmType; // BTree | Hnsw | Inverted | Graph |
    TimeSeries | Columnar
6
7     // 索引维护(由事务协调器在 commit / rollback 时调用)
8     fn insert(&self, tid: Tid, key: &Self::Key, txn: &Transaction) ->
    Result<()>;
9     fn delete(&self, tid: Tid, txn: &Transaction) -> Result<()>;
10
11     // 扫描
12     fn begin_scan(&self, predicate: &AmPredicate, snapshot: &Snapshot) ->
    Self::ScanState;
13     fn next(&self, state: &mut Self::ScanState) -> Option<Tid>;
14
15     // WAL 参与
16     fn wal_record_types(&self) -> &[WalRecordType];
17     fn redo(&self, record: &WalRecord) -> Result<()>;
```

```
18     fn undo(&self, record: &WalRecord) -> Result<()>;
19
20     // Checkpoint 参与
21     fn dirty_pages(&self) -> Vec<PageId>;
22     fn checkpoint_complete(&self, checkpoint_lsn: Lsn);
23
24     // GC 协作
25     fn reclaim_tid(&self, tid: Tid) -> Result<()>; // 收到 vacuum 通知后调用
26 }
```

每种访问方法自由决定内部存储格式，只要：

- 1. 通过 Layer 1 的 Page Allocator 申请页；
- 2. 变更时写 Layer 1 的 WAL；
- 3. 返回的结果是 TID，由 Layer 2 做可见性过滤。

深层架构创新

1. 分级同步模型（Tiered Freshness）

不是所有索引都需要在事务提交时同步更新。定义三级：

级别	含义	使用索引	延迟
Tier 0（同步）	事务提交前必须完成	主键 B + Tree，行存	0
Tier 1（事务内异步）	事务提交时批量合并	HNSW pre-tx delta，二级 B+Tree	0（对外）但写入路径可 batch
Tier 2（后台异步）	后台任务追赶，允许有界滞后	列存投影，全文倒排，时序聚合	秒~分钟级

 这很重要：如果 INSERT 一行要同步更新 6 个索引，写入吞吐会崩溃。Tier 2 异步让“写入路径”只需要：写 WAL + 更新行存 + 更新 B+ Tree + 合并 HNSW delta。全文，列存，时序后台追赶。

一致性保证：

- Tier 0 / 1:commit 后即可见(强一致)；
- Tier 2:提供 `await_index_fresh(index_name, target_lsn)` 接口，或查询时自动检测索引 watermark < 查询 snapshot LSN 时，fallback 到 seq scan 补全；

实现：

- **实现:**后台 worker 消费 WAL 流(类似 CDC consumer)，维护每个 Tier 2 索引的 `applied_lsn`。Visibility Oracle 暴露 watermark 信息，Planner 据此判断索引是否可用。

2. 统一的多路检索融合（Multi-Path Fusion）

Agent 的典型查询同时涉及结构化 + 向量 + 全文；

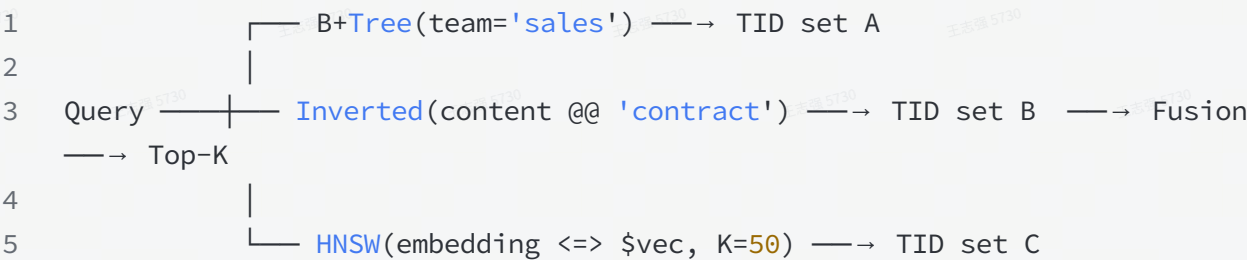
代码块

```
1 SELECT * FROM memory
2 WHERE team = 'sales' AND created_at > '2026-06-01'
3     AND content @@ 'contract'
4 ORDER BY embedding <=> $vec
5 LIMIT 10;
```

传统数据库做法是：优化器选 “最好的一条路径” （要么走索引 A，要么走索引 B）。

但混合检索需要多路并行 + 融合：

代码块



Fusion 策略：

- 过滤式： `A ∩ B ∩ C` (适合结构化/全文是硬约束)；
- RRF 排序式：各个路径排序后按 Reciprocal Rank Fusion 合并（适合软排序）；
- 混合式：结构化/全文作硬过滤，向量作排序。

架构要求：执行引擎（DataFusion）需要一个 `MultiIndexScan` 算子，能：

- 并行启动多个 AccessMethod scan；
- 按配置的 fusion strategy 合并 TID；
- 统一做 visibility check；
- 回表取完整行。

这不是 DataFusion 现有能力,需要自研算子。但 DataFusion 的 `ExecutionPlan` trait 足够灵活，可以插入自定义算子。

3. 行格式的“胖 header + 瘦 payload”设计

六种场景对行格式的需求：

- TP：定长列紧凑排列，快速随机访问；
- 向量：大块连续浮点数组，不适合和标量混存；
- JSONB：变长，可能很大；
- 图边：from / to / type / properties, properties 可能是 JSONB；
- 时序事件：timestamp + payload。

🏆 设计决策：主行存 slotted page 只存“胖 header + 定长标量列 + 列指针”，大对象 / 向量 / JSONB 存在 TOAST-like 的溢出页。

Tuple Layout(in slotted page):

TupleHeader(xmin/xmax/cid/lsn/agent_id/...) 【固定 ~48 bytes】

Fixed-width columns (INT, FLOAT, TIMESTAMP, ...) 【紧凑, O(1) 访问】

Varlen pointers(TOAST pointer for JSONB / VECTOR / TEXT) 【4-8 bytes each】

Overflow pages (TOAST):

VECTOR(1536) = 6144 bytes raw float32 【独立页】

JSONB payload 【独立页 (可跨多页)】

4. WAL 的“逻辑 + 物理”双模式

不同访问方法对 WAL 的需求不同：

- 行存 / B+Tree：需要物理 WAL (page-level redo / undo)，因为要精确恢复页状态；
- HNSW：更适合逻辑 WAL (record "add node X with vector V and neighbors [a,b,c]")，因为图结构重建比 page-level replay 更可靠；

- 列存投影：不需要自己的 WAL，从行存 WAL 派生即可（Tier 2 异步）；
- 全文倒排：逻辑 WAL（record：“add posting (term, tid)”）。

设计：WAL 记录分两类：

代码块

```
1  enum WalRecord {
2      // 物理记录:精确描述页变更(行存 / B+Tree 用)
3      Physical {
4          page_id: PageId,
5          offset: u16,
6          before_image: Vec<u8>,
7          after_image: Vec<u8>,
8      },
9
10     // 逻辑记录:描述高层操作(HNSW / 倒排 / 图 用)
11     Logical {
12         am_type: AmType,
13         operation: Vec<u8>, // operation 由 AM 自行序列化
14     },
15 }
```

恢复策略：

- Physical record：直接 redo / undo 到页；
- Logical record：调用对应 AM 的 redo() / undo() 方法，AM 自行决定如何重建。

好处：

- 行存 / B+Tree 用成熟的物理 WAL，性能确定；
- HNSW 等新型结构用逻辑 WAL，避免图状态的“部分页恢复”导致图不一致；
- 新增 AM 时，只需要实现 redo() / undo()，不需要理解物理页布局。

5. 跨模态 Cost-Based Optimizer

区别于 Postgres 最深也是最难的一层。

PG 的 planner 不知道的事：

- HNSW 近似 KNN 的代价（图遍历步数 + 距离计算）；
- BM25 评分的代价（postings 合并 + 评分）；
- 图遍历的代价（递归层数 + 边遍历）；
- 向量压缩（SQ/PQ）的代价（distortion error + 解码 CPU）。

PG 只能把 HNSW 当成「黑箱」,执行时 EXPLAIN 也只能给出「HNSW scan returned N rows」这种粒度,没法联合 cost model。

所以我们的 pg_rust 必须设计:

- 一个跨模态的成本模型,能同时估算 TP / AP / 向量 / 全文 / 图的代价;
- 能在统一 SQL 里做联合优化 ---- 比如 `ORDER BY embedding <=> $1 + ts_rank(...)` 的 RRF 融合,应该用 HNSW + 倒排的 candidate set 做加权融合,而不是分两个查询再 union;
- 优化器决策可解释(`EXPLAIN FORMAT JSON` 给出每条访问路径的代价分解);
- 对应 architecture.md § 4.2 的 RAG 检索流程:planner 拆分查询(结构化 → B+Tree / 时序 → 时序索引 / 全文 → 倒排 / 向量 → HNSW),各索引返回候选 TID,按 RRF 融合,再 MVCC 可见性过滤,最后回表。
- 这一层是 pg_rust 的真正杠杆点。PG 做不到,因为它的 optimizer 是「PG 内核 + 几个独立扩展」,跨扩展的 cost model 没法联合设计——pgvector 的作者在 PG 邮件列表里抱怨过这个问题,但 PG 扩展 API 的结构约束让它没法解决。

6. 单一 WAL + 单一 LSN + 单一事务

这是把「多模态」变成「统一内核」而不是「多引擎拼装」的根契约。

不管走哪种访问方法,所有变更都进同一个 append-only WAL,共享同一个单调 LSN 时钟:

代码块

```
1 LSN 1001: Insert into agent_memory
2 LSN 1002: HnswAddNode for the new embedding
3 LSN 1003: Update inverted index for 'contract dispute'
4 LSN 1004: Commit
```

具体落地:

- 每种访问方法有自己的 page format(HNSW 节点文件、postings file、邻接文件、时序分区)
- 但所有变更通过**统一的 WAL 记录类型**进入同一个 log
- 事务协调器对所有访问方法**统一 commit / rollback**

这个设计的代价:每加一个新访问方法(HNSW、倒排、图、时序),必须先回答三个问题——写哪种 WAL 记录、如何参与 checkpoint、如何 redo/undo。

这是 pg_rust 区别于「PG + pgvector + pgai + pg_textsearch 外挂拼接」的核心机制——后者每个扩展都有自己的 WAL(实际上是绕过 PG 的 WAL 直接写文件),崩溃恢复靠各自重建,事务原子性靠应用层补偿。

7. 多模态执行器: 行迭代 + 向量化 + 图遍历 混合

DataFusion 的向量化执行器对 AP 友好,对 HNSW(图遍历)和图(递归)不友好。一个统一执行器需要支持三种执行范式并能在同一查询里切换:

范式	适合场景	例子
行迭代 (Volcano)	TP, 图遍历, 点查	<code>SELECT * FROM orders WHERE id = 1</code>
向量化 (Arrow batch)	AP, 列存投影, 聚合	<code>SELECT region, sum(amount) FROM sales GROUP BY region</code>
图遍历 + 评分	向量 (HNSW), 全文 (BM25), 图 (Cypher)	<code>ORDER BY embedding <=> \$1 LIMIT 10</code>

执行器需要在 operator 层做模式切换——比如 RRF 融合算子能同时消费 HNSW 的图遍历结果和倒排的 postings list,并在中间用 Arrow batch 做融合。

当前 pg_rust 的状态:用 DataFusion 做执行器(见 planning.md § 六.4),但 DataFusion 是 AP-only(向量化),TP 路径用 Volcano,向量路径用 crate(HNSW 原型)。显式的「多范式执行器」架构没规划,这是需要补充和完善的。

8. 共享 Buffer Pool + 跨模态缓存

每种访问方法的 page format 不同,但都进同一个 buffer pool:

- 行存页(8KB)
- HNSW 节点页(变长)
- 倒排 postings 页
- 图邻接页
- 时序分区页

buffer pool 统一做:

- LRU/CLOCK 替换策略
- WAL 协调(page_lsn 与 WAL record_lsn 的对比)
- Checkpoint(同一帧可被不同访问方法复用)
- pin/unpin 协议(避免 evict 被使用中的页)

访问方法的差异在 page format 层抹平,统一在 frame 抽象里。

9. 一等公民类型 + 多模态 Schema

AGENT_ID / TRACE_ID / SESSION_ID / VECTOR(n) / TIMESTAMP 作为一等公民类型,自动参与:

- **provenance**:每行写入者身份,写入 Tuple Header(见 architecture.md § 3.1)
- **query trace**:SQL 生命周期记录,按 TRACE_ID 聚合
- **RLS 策略**:多 Agent 隔离(Phase 2+ 评估)
- **索引路由**:向量列自动用 HNSW、文本列自动用倒排、时间列自动用时序分区

这是 Agent Native 的元层能力——把「多模态」从「数据形态」提升到「类型系统」层级。

代码块

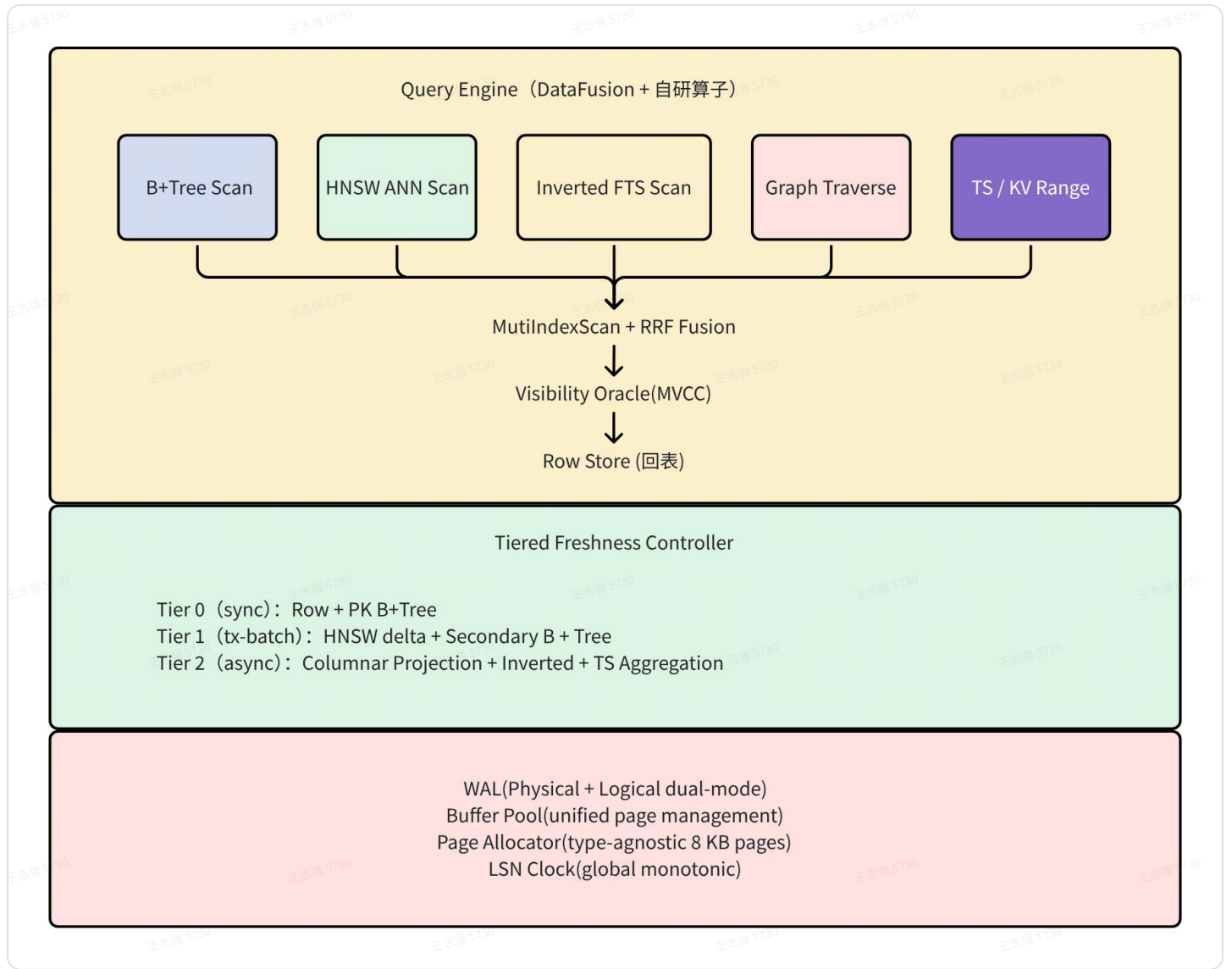
```
1 CREATE TABLE agent_memory (
2     id          TEXT PRIMARY KEY,
3     content     TEXT,
4     embedding   VECTOR(1536),
5     metadata    JSONB,
6     tags        TEXT[],
7     created_by  AGENT_ID,      -- 一等公民:Agent 身份
8     session_id  TRACE_ID,     -- 一等公民:会话追踪
9     created_at  TIMESTAMP DEFAULT now()
10 ) WITH (
11     vector_index = 'hnsw',
12     fulltext_index = 'bm25',  -- 同样声明式
13     ts_partition = 'day',     -- 同样声明式
14     provenance = true
15 );
```

这种 schema 声明式地把多模态索引语义化——`WITH (vector_index = 'hnsw', fulltext_index = 'bm25', ts_partition = 'day')` 不是 PG 的扩展 API 拼接,而是内核元数据,optimizer 一开始就知道有哪些索引可用。

当前 pg_rust 的状态:

-  DDL 示例已写
-  `VECTOR(n)` / `AGENT_ID` / `TRACE_ID` 类型已规划
-  `fulltext_index` / `ts_partition` 这种声明式元数据没显式规划
-  optimizer 自动路由(基于列类型 + WITH 子句)没规划

完整图景



架构合理性分析

1. 三层正交分离 (Layer 1/2/3) — 评价：极佳

这是文档最核心也最合理的设计。物理存储、可见性、访问方法三层解耦：

- Layer 1 的 "Page 不假设内容" 正是 PG 原始设计的精髓推广
- Layer 2 统一 MVCC 避免了每种 AM 重复实现事务语义 (PG 扩展的最大痛点)
- Layer 3 的 AccessMethod trait 提供清晰的可插拔契约

这种设计在学术上类似 Umbra/LeanStore 的分层，工程上比 PG 的 amapi.h 更干净。

2. 单一 WAL + 单一 LSN — 评价：正确且必要

这是区别于"多引擎拼装"的根本保证。崩溃一致性必须靠单一有序日志，这一点没有捷径。

3. 分级同步模型 (Tier 0/1/2) — 评价：务实

写路径只做 WAL + 行存 + B+Tree + HNSW delta，全文/列存/时序后台追赶。这解决了"写放大"问题，是 RocksDB/LSM 思路在多索引场景的合理推广。await_index_fresh() 接口提供了灵活的一致性选择。

4. WAL 逻辑+物理双模式 — 评价：成熟

行存用物理 WAL（确定性恢复），HNSW/倒排用逻辑 WAL（避免图状态部分页恢复）。这借鉴了 PG 的 Generic WAL + 逻辑复制的思路，但更系统化。

边界与风险

风险及设计的不够深入的部分：

1. 回表开销被低估

文档承认了 HNSW 回表问题（取 2x 候选），但对以下场景缺乏分析：

- 图遍历：多跳 BFS/DFS 每一步都回表做可见性检查，延迟会指数放大
- 全文检索：posting list 可能包含百万 TID，逐一回表不现实
- 缓解方案不够：文档提到“后续可升级为 TID+XID 索引条目”，但这实际上是在索引中嵌入 MVCC 信息，与“索引不管可见性”的设计原则矛盾，需要明确过渡路径

2. HNSW 并发控制缺失

HNSW 图结构的并发修改（插入新节点时修改邻居列表）是已知难题：

- Milvus 用段级别 seal+compact 避免原地修改
- 文档中 HNSW 参与事务 commit（Tier 1），但 HNSW 图结构不支持高效回滚
- undo() 对 HNSW 意味着什么？删除一个节点并修复所有邻居连接？这可能破坏图的连通性

3. 列存投影作为 Tier 2 的一致性窗口

AP 查询如果命中的是 Tier 2 列存投影，存在秒~分钟级的滞后：

- 文档提到 fallback 到 seqscan 补全，但这意味着 AP 查询可能退化为全表扫描
- 需要更精细的增量合并策略（类似 Delta Lake 的 Z-Order merge）

4. 跨模态 CBO 是“画饼”最重的部分

文档正确指出了问题（PG planner 不懂 HNSW/BM25 代价），但：

- 没有给出成本模型的数学公式或统计信息收集方案
- HNSW 的代价高度依赖数据分布（聚类程度），统计信息怎么维护？
- RRF 融合的代价估算本身就是开放研究问题
- 这是“真正的杠杆点”但也是最大的工程风险

5. Multi-Path Fusion 的正确性问题

RRF 融合（ $1/(k+\text{rank})$ 求和）在以下情况可能给出错误结果：

- 各路径返回的候选集不重叠时，RRF 退化为 union
- 结构化过滤是硬约束还是软约束？文档给了三种策略但没说 optimizer 如何自动选择

全面性分析

关键缺失

缺失领域	影响	说明
并发控制协议	致命	未说明用 2PL / OCC / SSI? 不同 AM 可能需要不同策略
死锁检测/预防	高	图遍历 + TP 操作可能产生复杂死锁图
内存管理	高	HNSW 需要大块连续内存, 与 Buffer Pool 的 8KB page 框架冲突
崩溃恢复流程	高	物理+逻辑 WAL 混合回放的顺序? 逻辑 WAL 的 redo 幂等性怎么保证?
Vacuum/GC 策略	高	6 种索引的 GC 协调复杂度远超 PG
统计信息收集	中	CBO 依赖统计, 向量/图/时序的统计信息形态未定义
错误处理模型	中	部分 AM 写入失败时如何回滚已写入的其他 AM?
测试/验证策略	中	如何验证 6 种 AM 的事务正确性? 需要形式化或 TLA+

次要缺失

缺失领域	说明
压缩策略	SQ/PQ 提了但没有与 Buffer Pool 的交互设计
复制协议	分布式是低优先级, 但单节点 WAL shipping 的设计也没提
连接协议	PG wire protocol 兼容? 还是自研?
Schema 演化	ALTER TABLE 对 6 种索引的 online DDL
资源隔离	HTAP 混合负载的 CPU/IO 隔离
向量量化	PQ/SQ 与 HNSW 的结合对可见性层的影响

与现有系统对比

对标系统	pg_rust 优势	pg_rust 劣势
PG + 扩展	统一事务、单一 WAL	扩展生态从零开始
SingleStore	类似 HTAP 思路	SingleStore 有 10 年工程积累
SurrealDB	类似多模态	SurrealDB 放弃了强一致性
GreptimeDB	类似时序+分析	GreptimeDB 不做向量/图
OceanBase	AI Native 方向一致	OceanBase 有千人团队

补充，完善及详细设计

1. 并发控制协议详细设计

推荐 SSI (Serializable Snapshot Isolation) ，与 MVCC 天然配合

设计目标与约束

目标：

- 默认隔离级别是 Snapshot Isolation，可选升级到 Serializable (SSI) ；
- 读不阻塞写，写不阻塞读 (MVCC 天然保证) ；
- 六种 Access Method 共享统一协议，不各自发明锁机制；
- 支持 Tier 0/1/2 分级同步下的正确性。

约束：

- 单节点，分布式事务暂不考虑；
- 单一 WAL + 单一 LSN 时钟；
- 所有索引通过 TID 回表做可见性判断；

分层并发控制

Layer4: 事务隔离层 SSI 冲突检测 / Write-Write 冲突 / 死锁检测
Layer3: 逻辑锁层 行锁 (Row Lock) 谓词锁 (Predicate Lock)
Layer2: 物理锁层

Page Latch / Buffer Frame Latch / Index Latch
Layer1: 原子原语层
Atomic / CAS / Mutex / RwLock / Epoch-based RCU

关键区分：Lock（逻辑锁） 跨事务、持续到事务结束；Latch（门锁） 保护内存结构、持续微秒级。

整体并发保证：

场景	协议	粒度	阻塞特征
行读	MVCC 快照	无锁	读永不阻塞
行写	Row Lock (X)	行	写-写互斥
B+Tree 读	Latch Coupling (R)	页	微秒级
B+Tree 写	乐观 / 悲观 Latch (W)	页	分裂时短暂
HNSW 读	Epoch pin (无锁)	无	永不阻塞
HNSW 写	Node-level fine-grained lock	节点	局部邻居锁
倒排读	Segment snapshot (无锁)	无	永不阻塞
倒排写	WriterBuffer Mutex	全局 buffer	极短
图读	Read Latch on adjacency page	页	微秒级
图写	Ordered node lock	节点对	双节点锁
时序读	分区不可变 (无锁)	无	永不阻塞
时序写	Active partition append	分区	无冲突
SSI 验证	SIRead + rw-conflict graph	谓词	仅 commit 时检查
死锁	Wait-for graph 周期检测	事务	100 ms 间隔检测

关键设计决策与权衡：

决策 1：默认 SI 而非 Serializable

- 理由：Agent 场景多为"读多写少"，SI 已避免脏读/不可重复读/幻读（大部分），Serializable 的 SSI 开销（谓词锁追踪）仅在需要时开启
- 代价：Write Skew 在 SI 下不可检测，需用户显式声明 SERIALIZABLE

决策 2: HNSW 用 Epoch + Node Lock 而非页级锁

- 理由: HNSW 的逻辑单元是"节点+邻居列表", 可能跨页; 页级锁会过度互斥
- 代价: 需要自定义内存管理, 不能完全复用 Buffer Pool 的 page latch

决策 3: Undo HNSW = 标记删除 + 后台修复

- 理由: 物理删除 HNSW 节点需要修复所有邻居的连通性, 代价太高且可能级联
- 代价: abort 后短暂的"幽灵节点"可能影响搜索精度 (但可见性过滤会排除它)

决策 4: 图遍历不加全路径锁

- 理由: 多跳遍历锁住整条路径会导致大量死锁和长时间持锁
- 代价: 图遍历结果在 SI 下是"快照一致"的, 但路径上的节点可能在遍历过程中被修改 (读已提交级别的 anomaly)

2. 崩溃恢复机制详细设计 (明确 REDO/UNDO 阶段如何处理混合 WAL)

设计前提与核心挑战

pg_rust 的崩溃恢复比传统数据库更复杂, 原因:

- 单一 WAL 中混合了物理记录 (行存 / B+Tree) 和逻辑记录 (HNSW / 倒排 / 图 / 时序);
- 六种 Access Method 的页格式不同, 恢复语义不同;
- Tier 0 / 1 / 2 分级同步意味着崩溃时各索引的推进程度不一致;
- HNSW 等图结构的 "部分写入" 可能导致结构性不一致 (不仅是数据不一致)。

恢复目标:

- 崩溃后数据库恢复到最后一个已提交事务的一致状态;
- 所有未提交事务的效果被完全撤销;
- Tier 2 索引允许滞后, 但必须能从已知的 watermark 点继续追赶;
- 恢复时间与脏页数量成正比, 不与数据库总大小成正比。

WAL 记录的分类与恢复语义

(1) 两种 WAL 记录

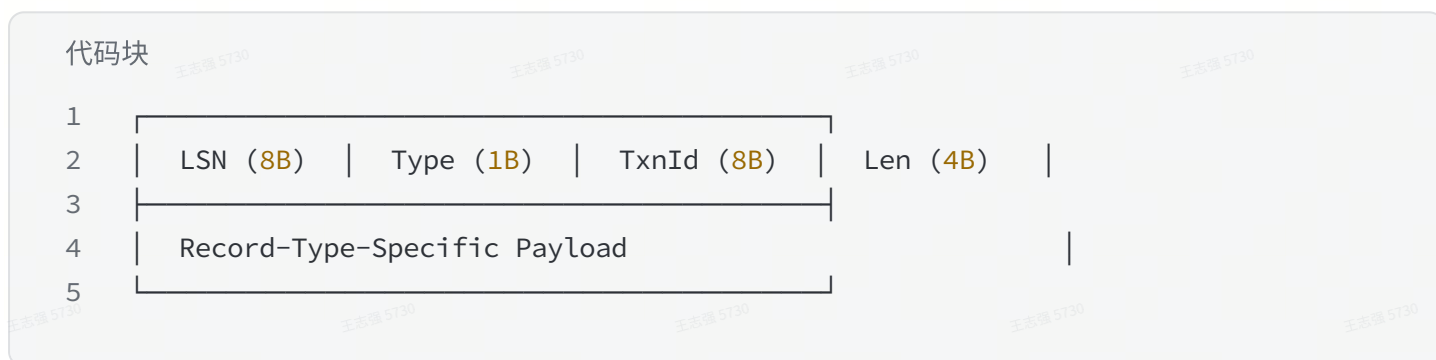
类型	使用者	内容	恢复方式
物理记录	行存堆表, B+Tree	精确的页内字节变更 (偏移 +before/after image)	直接覆盖写到对应页的对应位置
逻辑记录	HNSW, 倒排, 图, 时序	高层操作描述 (如 "添加节点 X, 邻居为 [a, b, c]")	调用对应的 AM 的 redo / undo 方法重建

(2) 为什么需要两种

- 物理记录的优势：恢复是确定性的、幂等的、不需要理解数据语义。对行存和 B+Tree 这种页结构成熟且固定的组件，物理记录是最高效最可靠的选择。
- 物理记录对 HNSW 的问题：HNSW 插入一个节点可能修改 10-30 个邻居节点的邻居列表，分布在多个页上。如果崩溃发生在这些页修改的中间：
 - 物理恢复能恢复已刷盘的页，但图的逻辑一致性（连通性、双向边对称）无法保证；
 - 你可能得到一个"页内容正确但图结构破碎"的状态。
- 所以 HNSW/图等结构用逻辑记录——记录的是"做了什么操作"，由 AM 自己决定如何从操作序列重建一致状态。

混合 WAL 的统一格式

每条 WAL 记录共享统一的 header：



Type 字段区分：

- LOGICAL_HNSW — HNSW 操作
- LOGICAL_INVERTED — 倒排索引操作
- LOGICAL_GRAPH — 图索引操作
- LOGICAL_TIMESERIES — 时序索引操作
- TXN_COMMIT — 事务提交标记
- TXN_ABORT — 事务回滚标记
- CHECKPOINT_BEGIN — 检查点开始
- CHECKPOINT_END — 检查点结束

Checkpoint 机制

恢复的起跑线，不需要回放整个 WAL 历史，只需要从最近的 checkpoint 开始。

- 模糊检查点 (Fuzzy Checkpoint)
 - 非阻塞检查点，不要求检查点时刻所有脏页都已经刷盘。
 - 流程：

- i. BEGIN_CHECKPOINT: 写一条 CHECKPOINT_BEGIN WAL 记录, 记录当前 LSN 为 ckpt_start_lsn
- ii. 收集元数据:
 1. 当前所有活跃事务列表 (ATT: Active Transaction Table)
 2. 所有脏页及其最早修改 LSN (DPT: Dirty Page Table)
 3. 每个 Tier 2 索引的 applied_lsn (watermark)
- iii. 后台刷脏页: Buffer Pool 逐步将脏页写入磁盘 (不阻塞正常操作)
- iv. END_CHECKPOINT: 所有脏页刷完后, 写 CHECKPOINT_END 记录, 包含 ATT 和 DPT 快照
- v. 更新超级块: 将 ckpt_start_lsn 写入数据库超级块 (磁盘固定位置), 标记"恢复从这里开始"

• 各 Access Method 参与 Checkpoint

AM	Checkpoint 时的职责
堆表	报告脏页列表, 配合 Buffer Pool 刷盘
B+Tree	同上
HNSW	报告脏的节点页; 额外写一条 "HNSW 元数据快照" (入口点、最大层数、节点计数)
倒排	将 write buffer 强制 flush 为 segment; 报告段文件列表
图	报告脏的邻接表页
时序	封存活跃分区为只读; 创建新活跃分区

- Checkpoint 频率策略
 - WAL 大小触发: WAL 自上次 checkpoint 后增长超过阈值 (默认 256MB)
 - 时间触发: 超过一定时间间隔 (默认 5 分钟)
 - 脏页比例触发: Buffer Pool 脏页超过 70%

崩溃恢复流程

崩溃重启后分为三个阶段:

Analysis (确定范围) -> Redo Phase (重做已提交) -> Undo Phase (撤销未提交)

经典 ARIES 的变体, 针对 pg_rust 的混合 WAL 做了扩展。

- 分析阶段 Analysis
- 重做阶段 Redo

- 撤销阶段 Undo

3. HNSW 并发协议 — 参考 Vamana/DiskANN 的无锁设计

// todo

4. 增量 Vacuum 协议 — 6 种 AM 的 GC watermark 推进机制

// todo

Roadmap

设计原则:

1. 每个阶段结束时，内部 Agent 团队都能用上 — 不是造完整个数据库才有价值
2. Layer 1 → Layer 2 → Layer 3 严格分层构建 — 地基不稳上层必塌
3. AM 按 Agent 记忆场景的优先级逐步加入 — 不追求一次性六种全做
4. 每阶段都有明确的正确性验证手段 — 数据库不能"大概能用"

Phase 0: 基座层 (Layer 1 核心)

目标：一个可靠的存储原语层，能分配页、写 WAL、管理缓存

交付物:

- Page Allocator (固定大小页分配/释放, freelist 管理)
- WAL Writer (append-only, fsync 语义, CRC 校验)
- Buffer Pool (page_id → frame 映射, LRU 替换, pin/unpin)
- LSN Clock (全局单调递增)
- Checkpoint Coordinator (脏页收集 + 刷盘 + WAL 截断)
- 崩溃恢复基础框架 (基于 WAL 的 redo 骨架)

验证标准:

1. 随机崩溃测试: 在任意时刻 kill 进程, 重启后数据完整
2. 性能基线: 顺序写 WAL 吞吐 $\geq 500\text{MB/s}$, Buffer Pool 随机读 $\geq 100\text{K ops/s}$
3. 单元测试覆盖率 $\geq 90\%$

价值: 一切的地基

Phase 1: 行存 + 事务 + B+Tree (最小可用数据库)

目标：能跑 CRUD 的单表关系存储，Agent 可以存结构化元数据

交付物：

- Heap Storage (slotted page 行存, TOAST 溢出页)
- Tuple 格式 (胖 header: xmin / xmax / agent_id / trace_id + payload)
- Transaction Manager (begin / commit / abort, MVCC 快照)
- Visibility Oracle (is_visible 判断)
- Lock Manager (行锁 + 死锁检测)
- B+Tree Index (latch coupling, 乐观插入)
- 崩溃恢复完整实现 (Analysis → Redo → Undo + CLR)
- Vacuum 基础版 (回收死元组, 通知索引清理)
- SQL 接口 (极简: CREATE TABLE / INSERT / SELECT / UPDATE / DELETE)
 - 基于 DataFusion 的 SQL 解析 + 执行

验证标准：

- ACID 正确性：并发读写 + 随机崩溃，数据始终一致
- 能跑简单的 Agent 元数据存取 (session 记录、用户画像、配置)
- TPC-C 简化版基准测试，确认 TP 性能不低于 SQLite 的 50%

对 Agent 团队的价值：

- 替代 SQLite/PG 存储 Agent 的结构化元数据
- session_id、agent_id、timestamp 等字段原生支持
- 行级 provenance (谁写的、什么时候写的)

Phase 2: HNSW 向量索引 (Agent 语义记忆)

目标：支持向量存储与近邻检索，Agent 可以做语义召回

交付物：

- VECTOR(n) 类型 (一定个公民, DDL 级声明)
- HNSW Access Method (实现 AccessMethod trait)
 - 构建 (分层图 + 随机层数 + 邻居选择)
 - 搜索 (贪心遍历 + ef 参数控制精度)
 - 并发 (epoch-based + 节点级锁)
- WAL 集成 (逻辑 WAL: add_node / remove_node)

◦ 崩溃恢复 (epoch snapshot + 增量重放 + 一致性修复)

- 向量距离函数 (L2 / Cosine / Inner Product, SIMD 加速)
- Tier 1 同步策略 (事务内批量合并 HNSW delta)
- 可见性过滤 (多取 2x 候选, 过滤后 top-K)
- SQL 语法扩展

◦ `ORDER BY embedding <=> $vector LIMIT K`

验证标准:

- 召回率: HNSW recall@10 $\geq$ 95% (标准 ANN benchmark 数据集)
- 正确性: 并发插入+删除+崩溃, 图结构始终一致
- 性能: 1M 768d 向量, 搜索延迟 < 10ms (ef_search=64)

对 Agent 团队的价值:

- Agent 对话/文档的 embedding 存储与语义检索
- 与结构化过滤组合: `WHERE team='sales' ORDER BY embedding <=> $v`
- 记忆的语义去重 (近似向量检测)

Phase 3: 全文倒排索引 (Agent 关键词记忆)

目标: 支持 BM25 全文检索, Agent 可以做关键词召回

交付物:

- TEXT 类型全文索引声明 (`WITH fulltext_index = 'bm25'`)
- Inverted Index Access Method
- 分词器 (中文 jieba + 英文 stemming)
- Posting List 存储 (segment-based, append-only)
- BM25 评分
- Tier 2 异步更新 (后台 worker 消费 WAL)
- Segment merge (后台 compaction)
- 全文检索语法 (`content @@ 'keyword'`)
- Watermark 机制 (planner 判断索引是否足够新鲜)

验证标准:

- 检索质量: 标准 IR 数据集 NDCG@10 对齐 Tantivy/ES 水平
- Tier 2 延迟: 写入后 < 1 秒可检索到 (正常负载下)
- 崩溃恢复: 从 applied_lsn 正确追赶

对 Agent 团队的价值：

- Agent 记忆的关键词检索 ("上次讨论过合同纠纷")
- 与向量检索互补 (向量找语义相似，全文找精确关键词)

Phase 4: Multi-Path Fusion (混合检索)

目标：单条 SQL 同时走多个索引并融合结果 — pg_rust 的核心差异化能力

交付物：

- MultiIndexScan 执行算子
 - 并行启动多个 AM scan
 - Fusion 策略 (过滤式 / RRF 排序式 / 混合式)
 - 统一 visibility check + 回表
- 跨模态 Planner (初版)
 - 识别查询中的多模态谓词
 - 选择 fusion 策略
 - 基础 cost 估算 (各 AM 的选择性估计)
- SQL 语法
 - 自然表达：WHERE + ORDER BY 组合自动触发多路融合

示例查询：

代码块

```
1  SELECT * FROM agent_memory
2  WHERE team = 'sales' -- B+Tree
3  AND content @@ 'contract' -- 倒排
4  ORDER BY embedding <=> $vec -- HNSW
5  LIMIT 10;
```

验证标准：

- 正确性：融合结果与"分别查询再应用层合并"结果一致
- 性能：多路并行 < 各路串行之和的 60%
- 质量：RRF 融合的 NDCG 优于单路最佳

对 Agent 团队的价值：

- Agent 一次"回忆"调用 = 一条 SQL，同时利用语义+关键词+结构化
- 大幅简化 Agent 记忆召回的应用层代码

- 这是对外推广时最强的 demo 场景

Phase 5: 图索引 + 时序索引（记忆关联与时间线）

目标：支持知识图谱式记忆关联 + 时间衰减式记忆管理

交付物：

- Graph Access Method
 - 邻接表存储
 - 多跳遍历（BFS/DFS，深度限制）
 - 路径查询语法（类 Cypher 或 SQL 扩展）
 - 并发协议（有序节点锁）
- TimeSeries Access Method
 - 时间分区（自动按天/小时）
 - 范围扫描 + 降采样
 - TTL 自动过期（记忆遗忘）
 - 分区 compaction
- 记忆生命周期管理
 - 遗忘曲线（基于访问频率 + 时间衰减）
 - 记忆蒸馏（多条细节 → 一条摘要，保留图关系）
 - 记忆分层（短期 → 工作 → 长期，自动晋升/淘汰）
- 图 + 时序参与 Multi-Path Fusion

对 Agent 团队的价值：

- "用户 A 上周投诉了物流 → 关联到订单 X" — 图查询
- "最近 7 天的所有交互按时间线展示" — 时序查询
- 自动遗忘不重要的记忆，保留关键记忆 — TTL + 蒸馏

Phase 6: 协议层 + 对外接口

目标：让外部 Agent 框架和工具能连接使用

交付物：

- PG Wire Protocol（兼容 psql / 主流 PG 驱动）
- MCP Server（原生提供记忆读写的 MCP 工具）
- REST API（轻量 HTTP 接口，嵌入式场景）

- 连接池 + 会话管理
- 基础权限体系（Agent 级别的 RLS）

对外推广前提：

- Phase 1-5 的功能稳定运行 3 个月+
- 内部 Agent 日常使用无重大问题
- 有 2-3 个完整的 Agent 记忆场景 demo

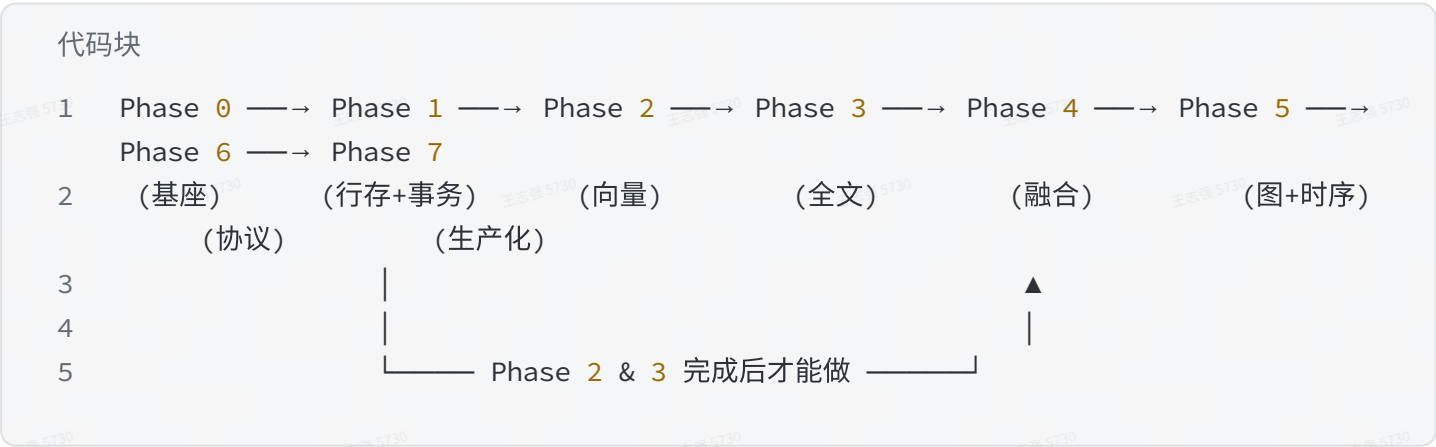
Phase 7：生产化 + 性能优化

目标：达到可对外推广的生产质量

交付物：

- 列存投影（AP 场景，记忆分析/统计）
- 向量压缩（SQ/PQ，降低存储成本）
- 完整的 Cost-Based Optimizer（跨模态代价模型）
- 监控 / 可观测性（EXPLAIN 跨模态分解）
- 备份 / 恢复 / WAL shipping
- Jepsen 级别正确性测试
- 性能调优（SIMD、io_uring、大页内存）

依赖关系



Milestones

Milestone	Agent 可以使用的能力
Phase 1	结构化元数据存取，替代 SQLite
Phase 2	语义记忆检索，替代外挂向量库

Phase 3	关键词记忆检索，无需外接 ES
Phase 4	一条 SQL 完成混合召回（核心 demo 场景）
Phase 5	完整的记忆生命周期管理
Phase 6	可以对外提供服务

风险控制

1. Phase 1 是最关键的 — 事务 + 崩溃恢复的正确性决定一切，宁可多花时间也不能赶
2. Phase 2 是最有 demo 价值的 — 内部 Agent 最先需要向量检索，尽早交付
3. Phase 4 是对外差异化的核心 — 这是 PG 扩展生态做不到的事，是推广时的杀手锏
4. 每个 Phase 结束时写一篇内部技术博客 — 为未来对外推广积累素材

References

- [OceanBase AI Native 的数据库](#)
- [数据库厂商下一个十年的入场券](#)